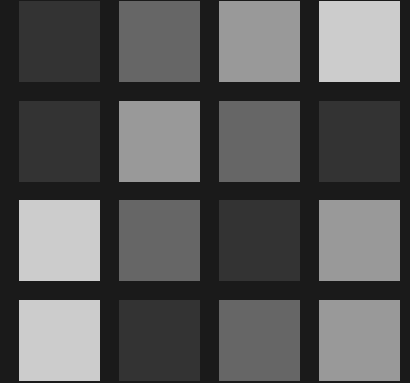




COMPUTER VISION

From Fundamentals to Generative AI & GANs

*Foundations of image understanding, deep learning for vision, and an
introduction to generative models — with runnable Google Colab notebooks*

Four Parts, One Story: Seeing → Understanding → Creating

01

Foundations

What images are made of, and how classical CV reads them

02

Deep Learning for Vision

CNNs, landmark architectures, and transfer learning

03

Advanced CV Tasks

Detection, segmentation, and vision transformers

04

Generative AI & GANs

Autoencoders, GANs, and where generative AI is headed

What We'll Cover, Slide by Slide

01

Foundations

- What Is Computer Vision?
- Real-World Applications
- Anatomy of a Digital Image
- Preprocessing — Colab
- Classical CV Techniques

02

Deep Learning for Vision

- Convolutional Neural Networks
- Image Classification — Colab (Keras)
- Kaggle → Classifier: Steps + Colab
- Landmark Architectures
- Transfer Learning
- Object Detection
- Image Segmentation
- Vision Transformers

03

Generative AI & GANs

- What Is Generative AI?
- Autoencoders
- Generator vs Discriminator
- How GANs Train
- Build a Simple GAN — Colab
- GAN Variants
- GANs in Action
- Beyond GANs: Diffusion

PLUS: KEY TAKEAWAYS, A PRACTICE SELF-CHECK, AND FURTHER RESOURCES TO CLOSE



PART 01

Foundations of Computer Vision

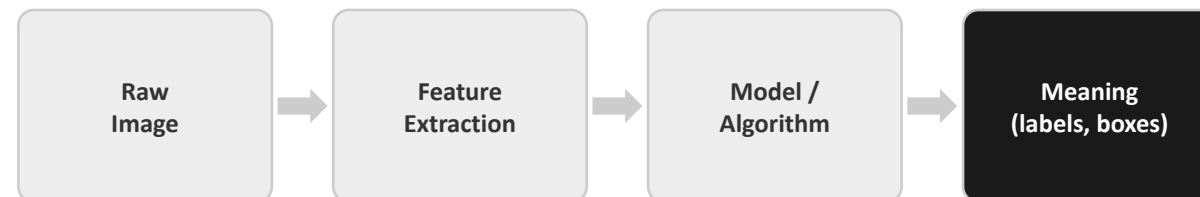
What machines see, and how they start to make sense of it

What Is Computer Vision, in Plain Language?

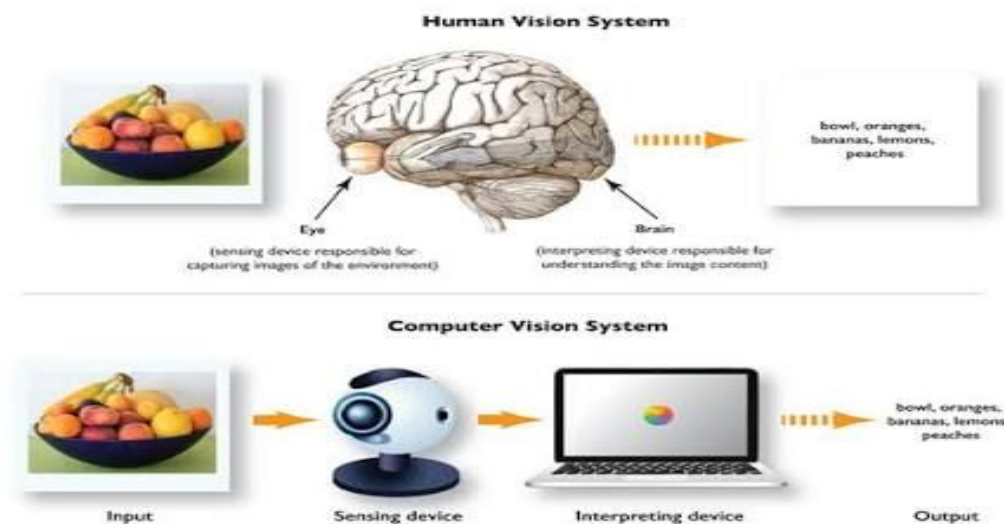
Computer Vision is the field that teaches software to make sense of pictures and video the way a person glances at a photo and instantly knows what's in it. A computer never “sees” a cat or a car — it only ever receives a grid of numbers. Computer Vision is the set of techniques that turn that grid of numbers into a useful answer: “this is a cat,” “this object is 3 metres away,” or “this part of the image is the road.”

Think of it as reading, but for pixels instead of words.

A COMPUTER VISION PIPELINE, AT ITS SIMPLEST



Every technique in this deck — from a 1990s edge detector to a modern GAN — fits somewhere in this same pipeline.



Where This Shows Up: Everyday Scenarios



Hospital Radiology

A model flags a suspicious region on an X-ray before a radiologist reviews it.



Highway Driving

A car's camera tells the difference between a pedestrian, a cyclist, and a shadow, in real time.



Retail Checkout

A camera recognises items placed in a basket so no barcode scan is needed.



Airport Security

A scanner highlights unusual shapes inside a bag for a human officer to check.



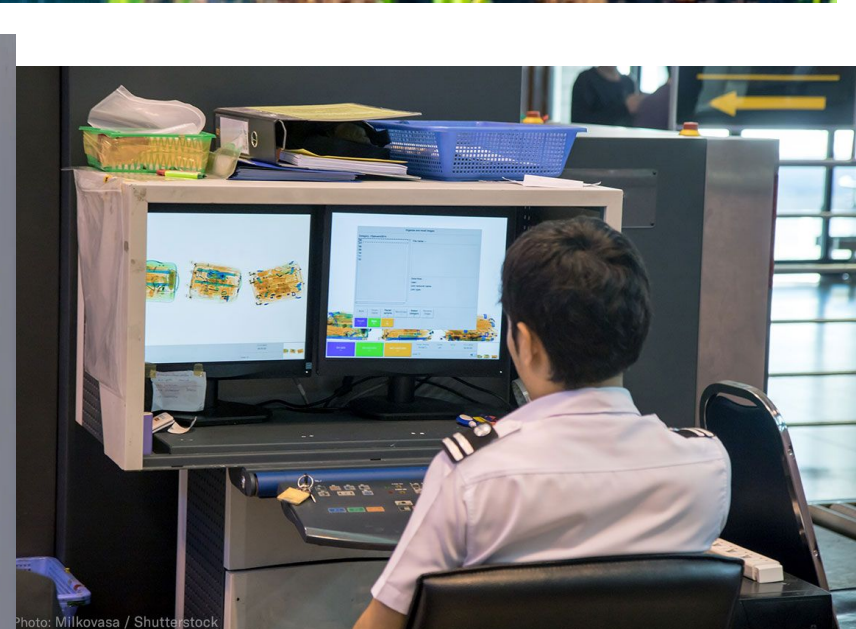
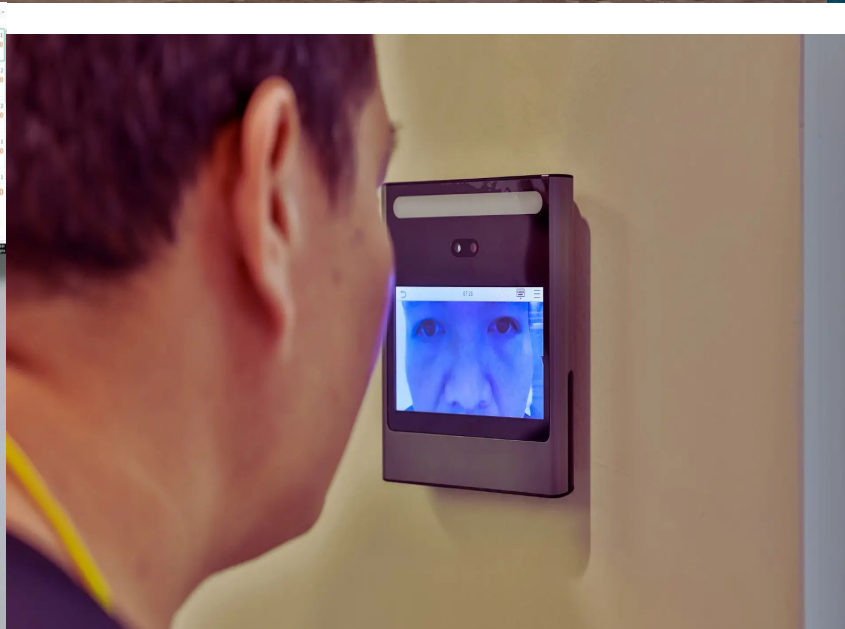
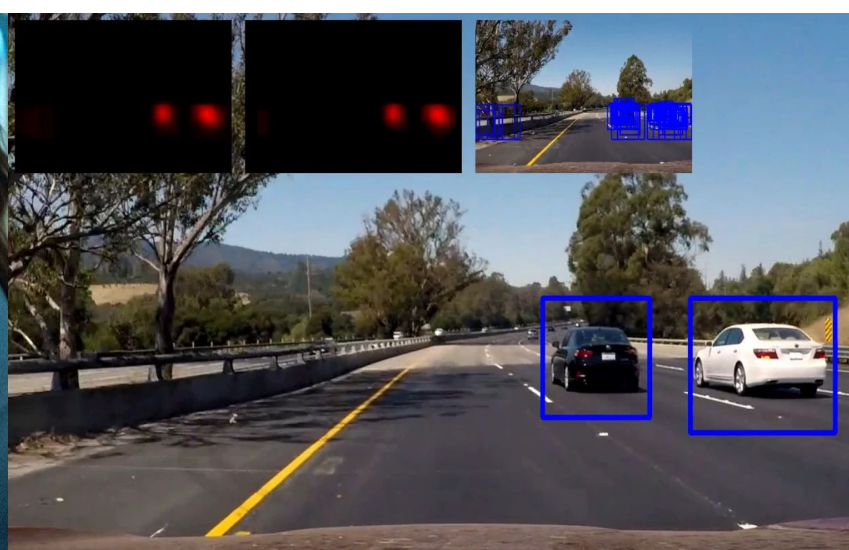
Agriculture Drones

Aerial images are scanned to spot crop disease across thousands of acres.



Social Media

A photo app suggests captions and blurs faces of people who asked to opt out.



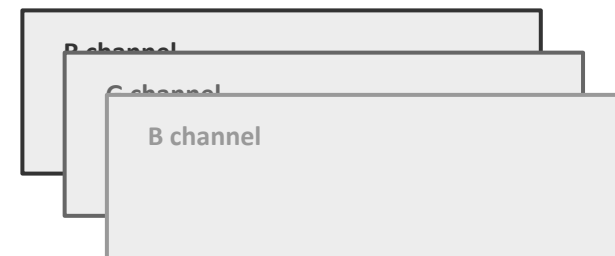
Anatomy of a Digital Image

Every image a computer works with is really a matrix of numbers. A grayscale image is one grid of brightness values (0 = black, 255 = white). A colour image is three such grids stacked together — one each for Red, Green, and Blue — combined at every pixel to produce the colour you see.

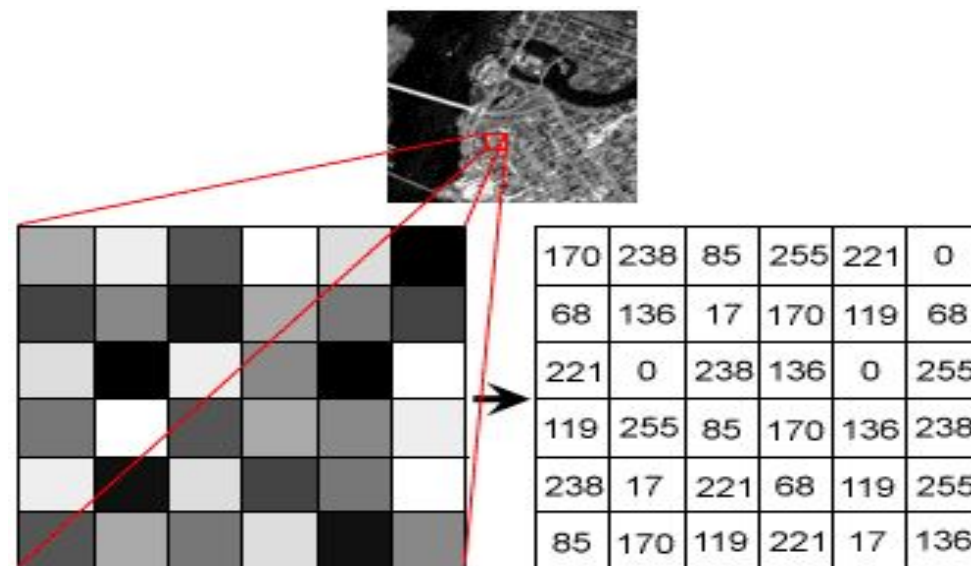
- A 1920×1080 photo is over 2 million pixels
- Each RGB pixel is 3 numbers, 0–255
- Models usually rescale these to 0–1 first

GRAYSCALE PIXEL GRID

12	240	88	5
200	33	250	120
64	190	10	255
8	150	200	40



A COLOUR IMAGE = 3 STACKED CHANNELS



Preparing Images Before a Model Ever Sees Them

Raw images arrive in inconsistent sizes, brightness levels, and formats. Preprocessing standardises all of that — resizing every image to the same dimensions, scaling pixel values to a small range, and sometimes creating slightly altered copies (flips, rotations) so the model generalises better.

GOOGLE COLAB

```
# Runs directly in Google Colab — no local setup needed
import cv2, numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread('sample.jpg')
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (224, 224))          # standard input size
img = img.astype(np.float32) / 255.0      # scale to 0-1

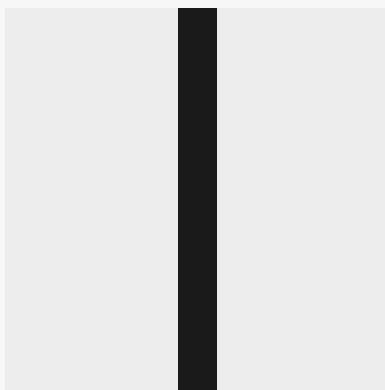
print('Shape after preprocessing:', img.shape)
cv2_imshow((img * 255).astype(np.uint8))
```

Classical Computer Vision: Rules Before Learning

Before deep learning, engineers hand-designed rules to find useful patterns in pixels. Two ideas still worth knowing:

EDGE DETECTION

Sobel / Canny filters

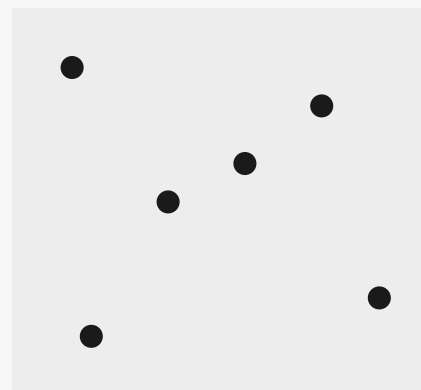


A sharp change in brightness between neighbouring pixels is treated as an edge.

Used for: contour detection, document scanning, lane-line detection.

FEATURE EXTRACTION

SIFT / HOG descriptors



Distinctive points (corners, blobs) are turned into numeric descriptors that survive rotation and scale changes.

Used for: panorama stitching, object matching, early face recognition.

EDGE DETECTION

Sobel / Canny filters

```
import cv2
import matplotlib.pyplot as plt

# Load image in grayscale
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply Filters
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)

canny = cv2.Canny(img, 100, 200)

# Display Results
plt.figure(figsize=(10, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap='gray');
plt.title('Original')
plt.subplot(1, 3, 2); plt.imshow(sobel, cmap='gray');
plt.title('Sobel')
plt.subplot(1, 3, 3); plt.imshow(canny, cmap='gray');
plt.title('Canny')
plt.tight_layout()
plt.show()
```

```
import cv2
import matplotlib.pyplot as plt
from skimage.feature import hog as sk_hog # Used only for clean HOG
visualization
```

```
# Load image
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert for
Matplotlib
```

```
# --- SIFT Visualization ---
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
sift_img = cv2.drawKeypoints(img_rgb, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

```
# --- HOG Visualization ---
# visual_blur extracts the gradient orientation lines to plot
_, hog_img = sk_hog(gray, visualize=True, pixels_per_cell=(16, 16))
```

```
# Display Results
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(sift_img); plt.title('SIFT Keypoints')
plt.subplot(1, 2, 2); plt.imshow(hog_img, cmap='gray'); plt.title('HOG
Gradients')
plt.tight_layout()
plt.show()
```

Why Hand-Designed Rules Ran Out of Road

- A hand-written edge rule that works on one photo often fails on the next — different lighting, angle, or background breaks it.
- Real-world categories (“cat,” “pothole,” “tumour”) have far too much visual variety to describe with fixed rules.
- Around 2012, deep learning showed it could learn these rules from data instead — and consistently beat hand-crafted methods.

HAND-DESIGNED RULES

“If the pixel gradient exceeds X , call it an edge...”



LEARNED REPRESENTATIONS

“Show the network 1M labelled images — let it discover the rules.”



PART 02

Deep Learning for Vision

Letting the network learn its own features, layer by layer

Convolutional Neural Networks (CNNs)

A CNN slides small filters across the image, each one learning to detect a simple pattern — an edge, a texture, a colour blob. Stacking many layers lets early filters detect simple shapes while deeper layers combine them into whole objects.



Filters in early layers learn simple things like edges and colour patches. Filters in deeper layers combine those into eyes, wheels, or fur — without anyone hand-coding what an “eye” looks like.

Convolution Neural Network (CNN)

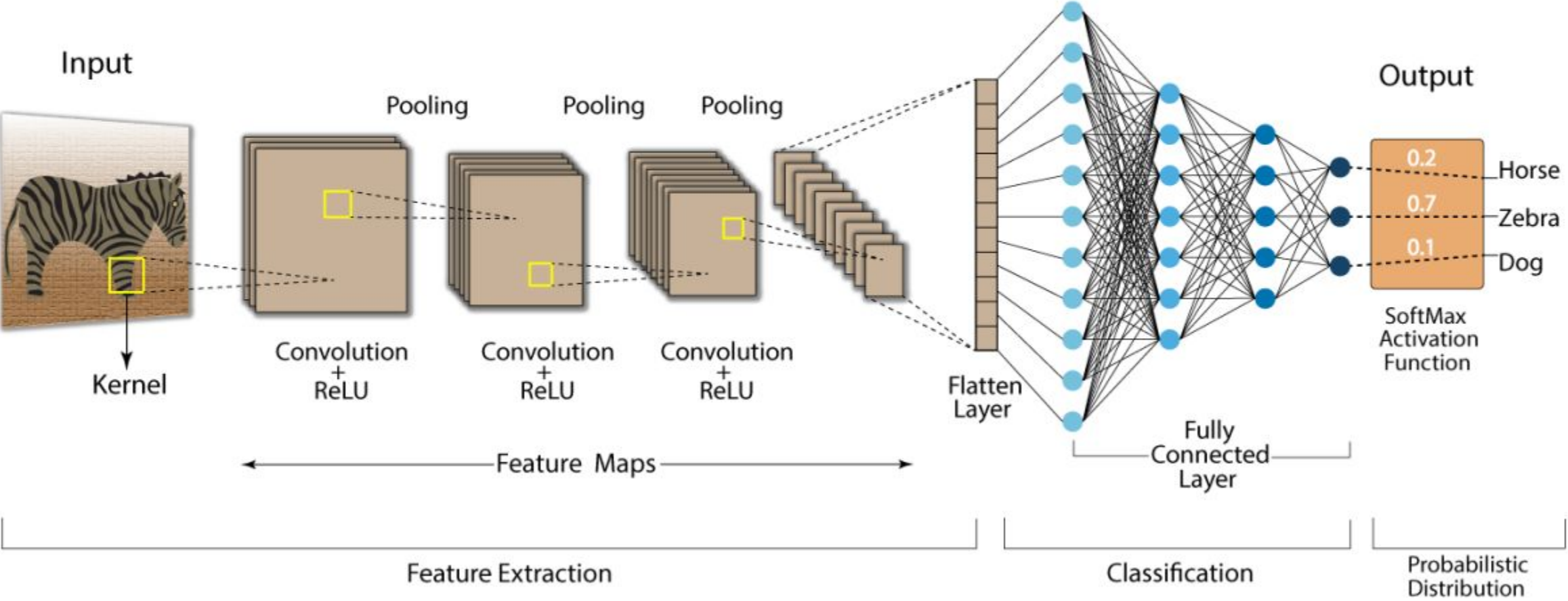


Image Classification with TensorFlow & Keras

The same CNN idea, expressed in Keras' high-level API: stack layers declaratively, compile with a loss and optimiser, then call `.fit()`. This is the fastest path from raw folders of images to a trained classifier.

GOOGLE COLAB

```
import tensorflow as tf
from tensorflow.keras import layers, models

num_classes = 5
model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(180, 180, 3)),
    layers.Conv2D(32, 3, activation='relu'), layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'), layers.MaxPooling2D(),
    layers.Conv2D(128, 3, activation='relu'), layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes, activation='softmax'),
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(train_ds, validation_data=val_ds, epochs=10)
```

From Kaggle to Classifier, Step by Step

1

Create a Kaggle API token: Kaggle account → Settings → Create New Token. This downloads kaggle.json.

2

In Colab, upload kaggle.json and move it where the Kaggle CLI expects it (~/.kaggle/).

3

Download and unzip the dataset directly into the Colab runtime using the Kaggle CLI.

4

Point `tf.keras.utils.image_dataset_from_directory` at the unzipped folder to build train/validation pipelines.

5

Feed those datasets straight into the Keras classifier's `.fit()` — no manual image loading needed.

Kaggle Dataset → Trained Classifier — in Colab

GOOGLE COLAB

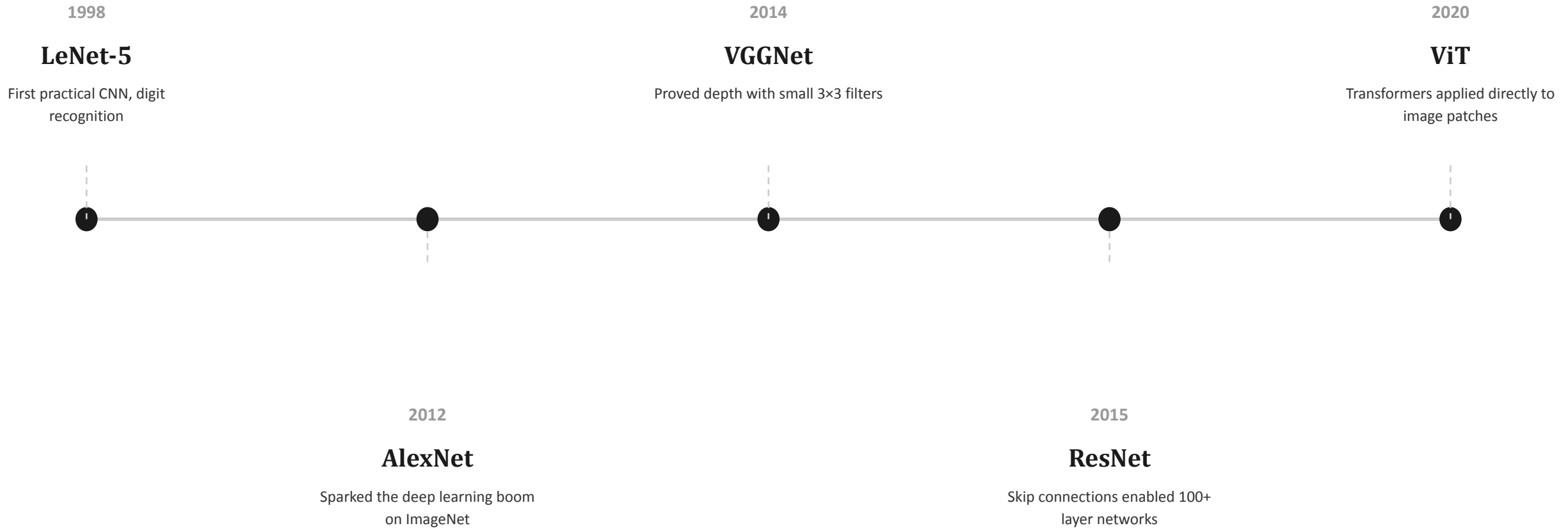
```
# Step 1-2: upload kaggle.json, then place it where the CLI expects it
from google.colab import files
files.upload()                # select kaggle.json
!mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
```

```
# Step 3: download and unzip a dataset (example: cats vs dogs)
!kaggle datasets download -d salader/dogs-vs-cats
!unzip -q dogs-vs-cats.zip -d dataset
```

```
# Step 4: build train/validation pipelines straight from folders
train_ds = tf.keras.utils.image_dataset_from_directory(
    'dataset/train', image_size=(180, 180), batch_size=32)
val_ds = tf.keras.utils.image_dataset_from_directory(
    'dataset/test', image_size=(180, 180), batch_size=32)
```

```
# Step 5: train the classifier defined on the previous slide
history = model.fit(train_ds, validation_data=val_ds, epochs=10)
```

Landmark CNN Architectures, at a Glance



```
graph BT; L1[11 x 11 Conv (96), stride 4] --> L2[3 x 3 MaxPool, stride 2]; L2 --> L3[5 x 5 Conv (256), pad 2]; L3 --> L4[3 x 3 MaxPool, stride 2]; L4 --> L5[3 x 3 Conv (384), pad 1]; L5 --> L6[3 x 3 Conv (384), pad 1]; L6 --> L7[3 x 3 Conv (384), pad 1]; L7 --> L8[3 x 3 Conv (384), pad 1]; L8 --> L9[3 x 3 MaxPool, stride 2]; L9 --> L10[3 x 3 Conv (384), pad 1]; L10 --> L11[3 x 3 Conv (384), pad 1]; L11 --> L12[3 x 3 Conv (384), pad 1]; L12 --> L13[3 x 3 MaxPool, stride 2]; L13 --> L14[FC (4096)]; L14 --> L15[FC (4096)]; L15 --> L16[FC (1000)];
```

Diagram illustrating the AlexNet architecture, showing the flow of data through various layers:

- 11 x 11 Conv (96), stride 4
- 3 x 3 MaxPool, stride 2
- 5 x 5 Conv (256), pad 2
- 3 x 3 MaxPool, stride 2
- 3 x 3 Conv (384), pad 1
- 3 x 3 Conv (384), pad 1
- 3 x 3 Conv (384), pad 1
- 3 x 3 Conv (384), pad 1
- 3 x 3 MaxPool, stride 2
- 3 x 3 Conv (384), pad 1
- 3 x 3 Conv (384), pad 1
- 3 x 3 Conv (384), pad 1
- 3 x 3 MaxPool, stride 2
- FC (4096)
- FC (4096)
- FC (1000)

Diagram illustrating a sequential neural network architecture:

- Top layer: 2×2 MaxPool, stride 2
- Second layer: 3×3 Conv, pad 1
- Third layer: ...
- Bottom layer: 3×3 Conv, pad 1

Arrows indicate the flow of data from the bottom layer up to the top layer.

[illegible]

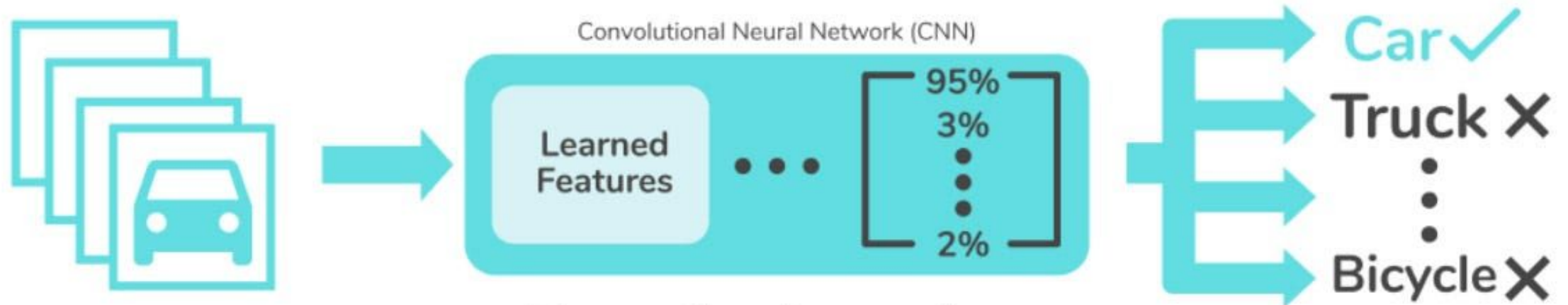
Transfer Learning: Reuse Before You Retrain

Training a CNN from zero needs millions of images. Transfer learning instead starts from a network already trained on a huge dataset (like ImageNet), keeps its learned features, and only retrain the last layer for the new task — often with just a few hundred images.



The grey box (frozen) keeps its weights unchanged. Only the black box (new head) learns your classes — often in just a few training epochs.

Training from Scratch



Transfer Learning



Object Detection: Where, Not Just What

Classification answers “what is in this image?”. Detection goes further — it locates every object with a bounding box and labels each one. Modern detectors like YOLO and Faster R-CNN do this in a single pass over the image, fast enough for live video.

Family	Examples	Style
Two-stage	Faster R-CNN	Propose regions, then classify — accurate, slower
One-stage	YOLO, SSD	Predict boxes + classes together — fast, real-time

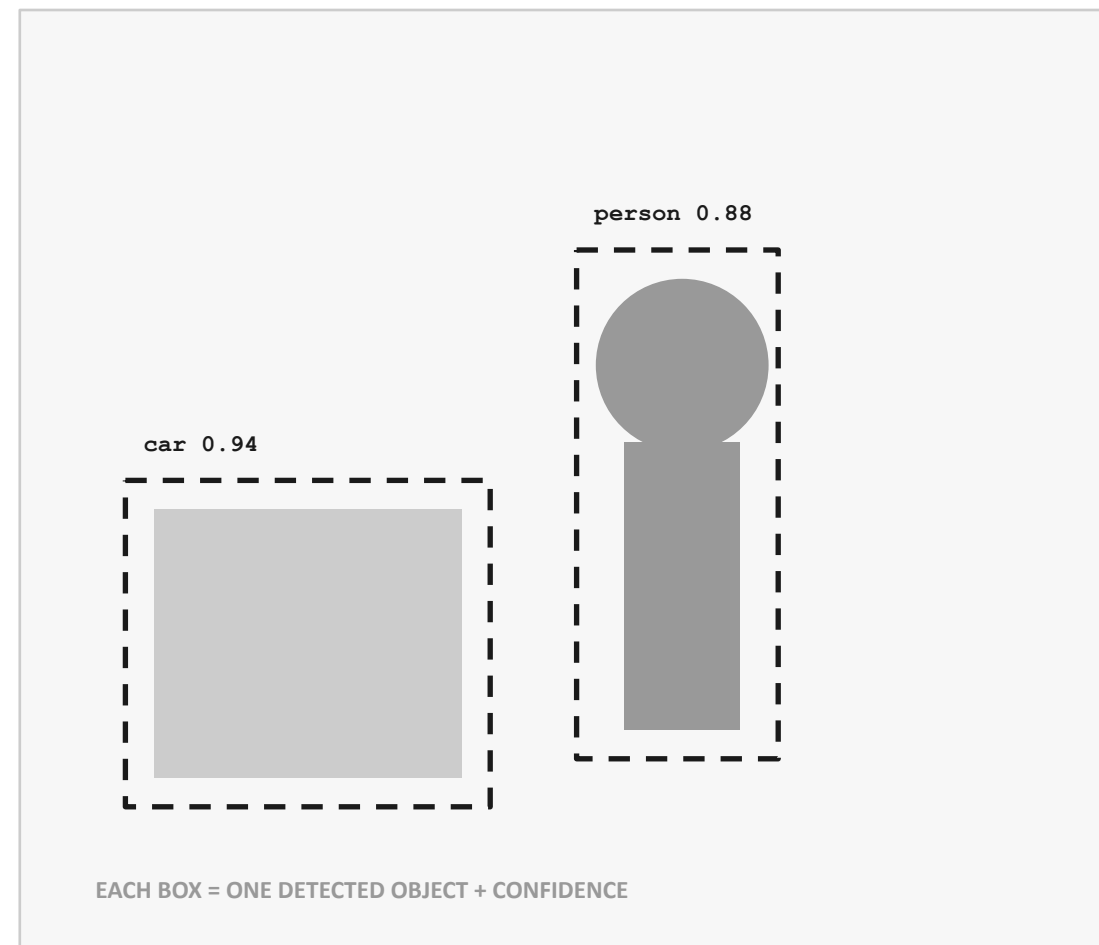
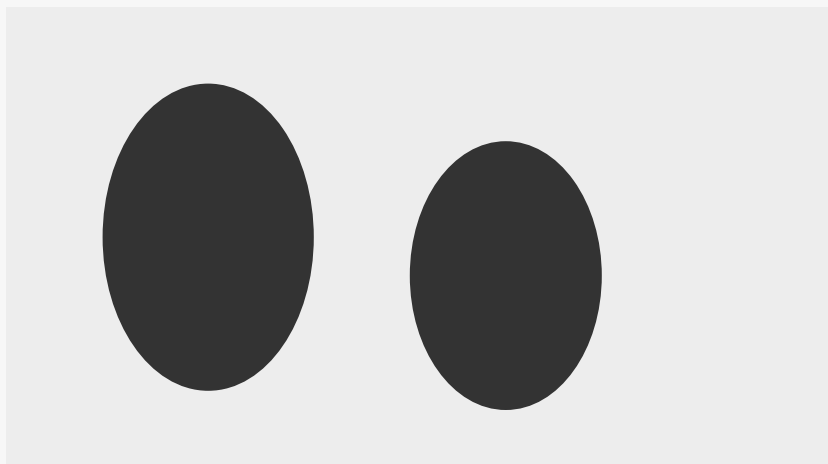


Image Segmentation: Pixel-Level Understanding

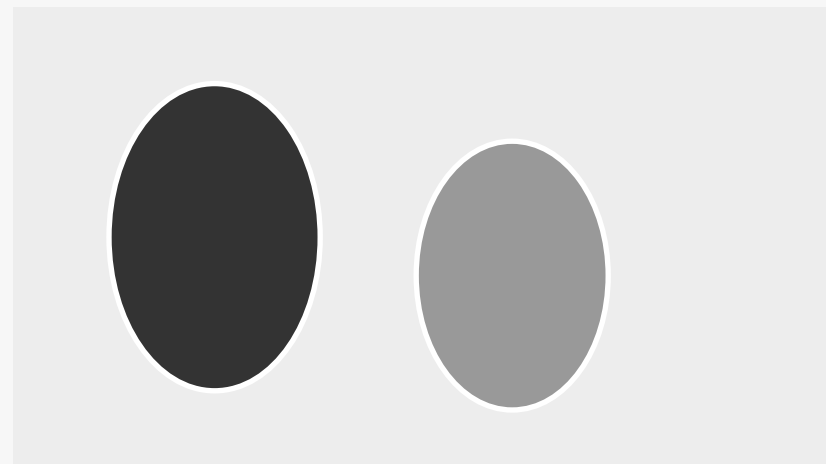
Segmentation labels every single pixel rather than drawing a box around an object — useful whenever the exact shape matters, like measuring a tumour's boundary or letting a car know exactly where the road ends.

SEMANTIC SEGMENTATION



Both objects share the SAME label: “person” — no distinction between individuals.

INSTANCE SEGMENTATION



Each object gets its OWN identity: “person #1”, “person #2” — separately outlined.

ORIGINAL IMAGE

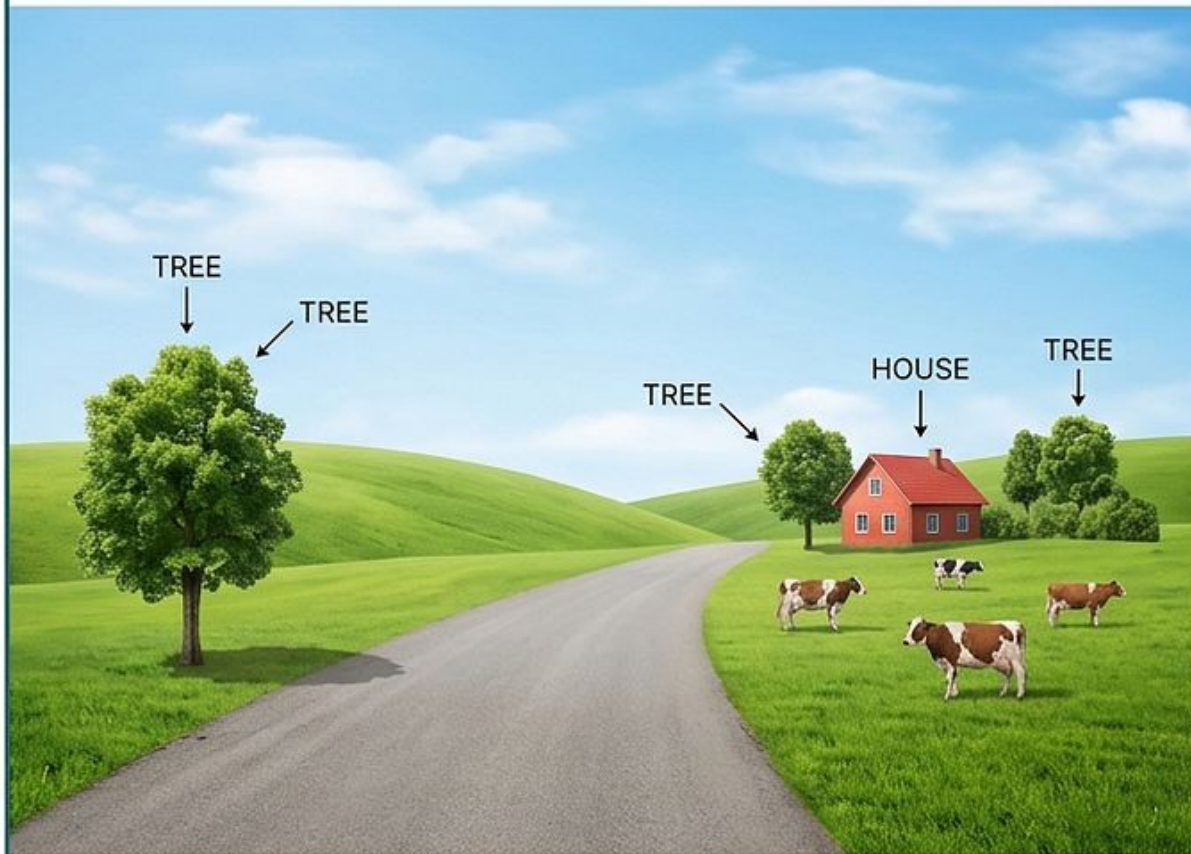
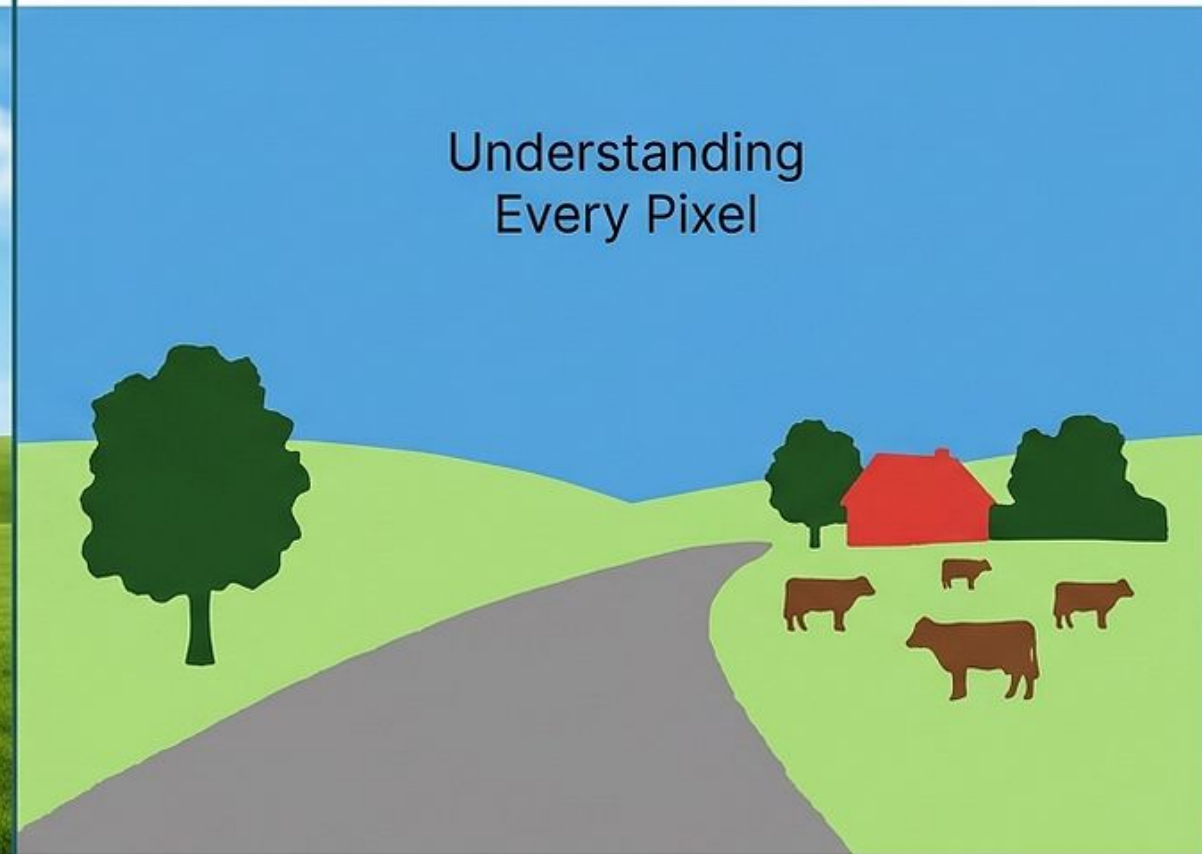
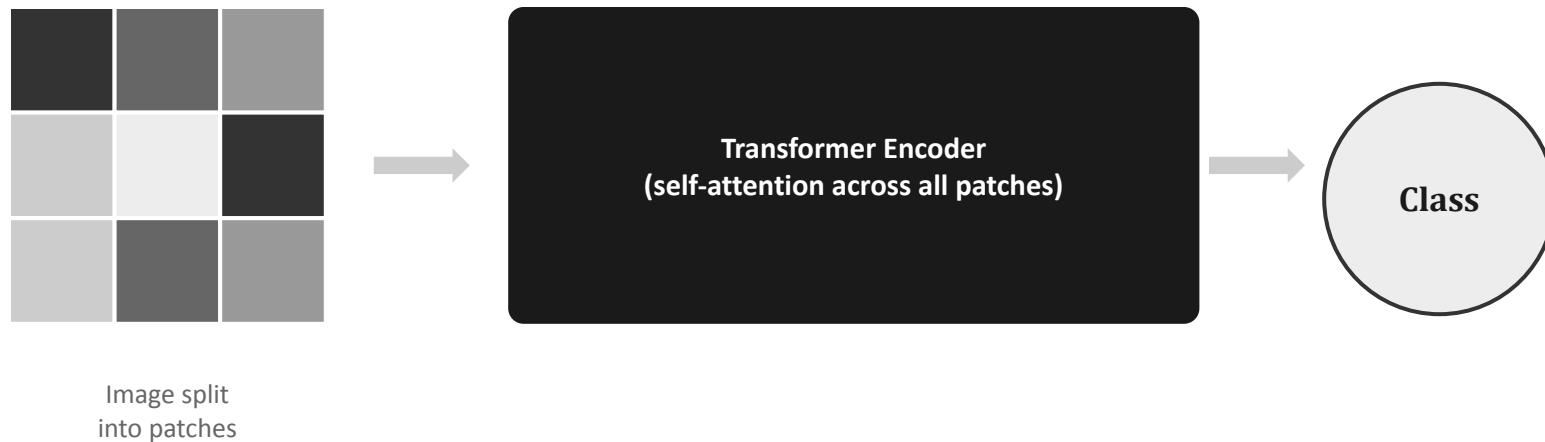


IMAGE SEGMENTATION

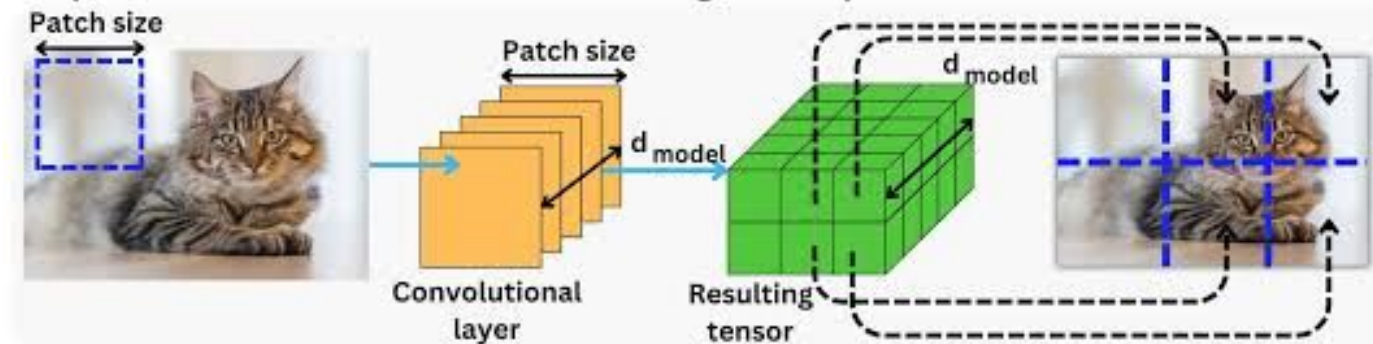


Vision Transformers (ViT): Patches Instead of Pixels

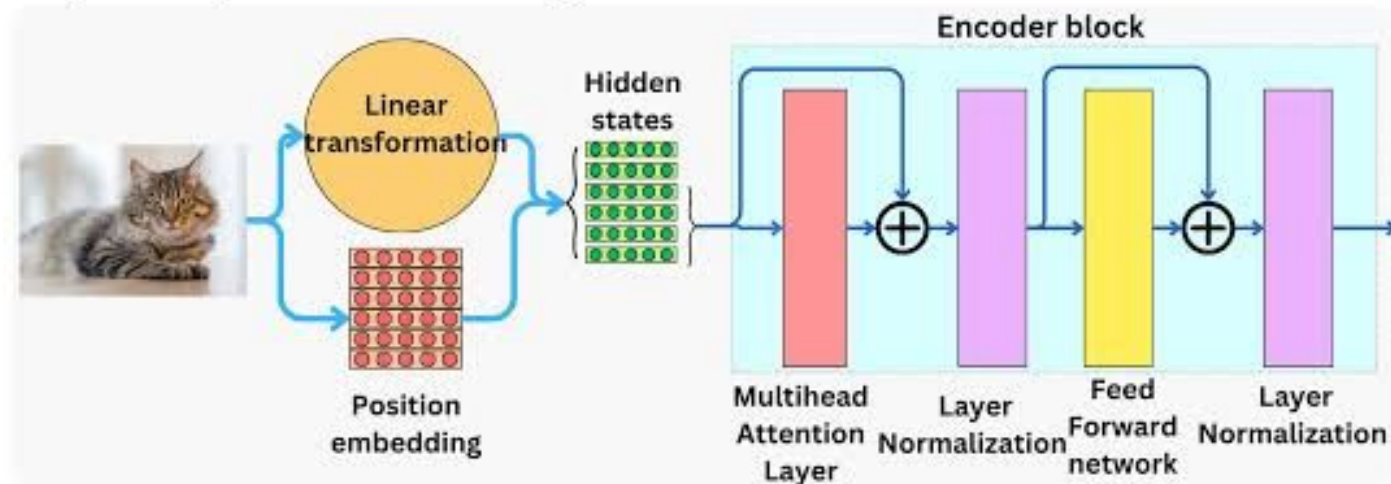
Transformers, originally built for language, were adapted to vision by a simple trick: cut the image into fixed-size patches, treat each patch like a “word,” and let self-attention learn which patches relate to which. With enough data, ViT models match or beat CNNs on many benchmarks.



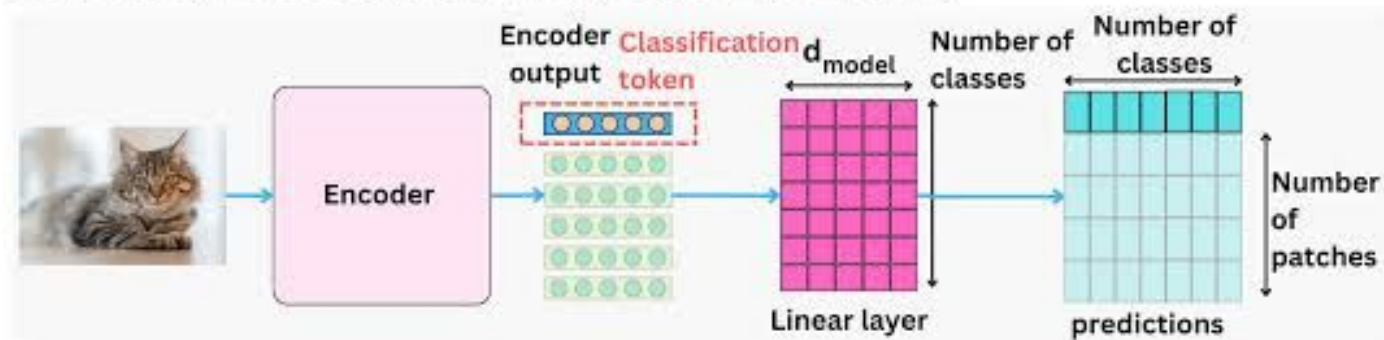
Step 1: Linear transformation from Image to sequence of vectors



Step 2: Add position embedding and Encoder blocks



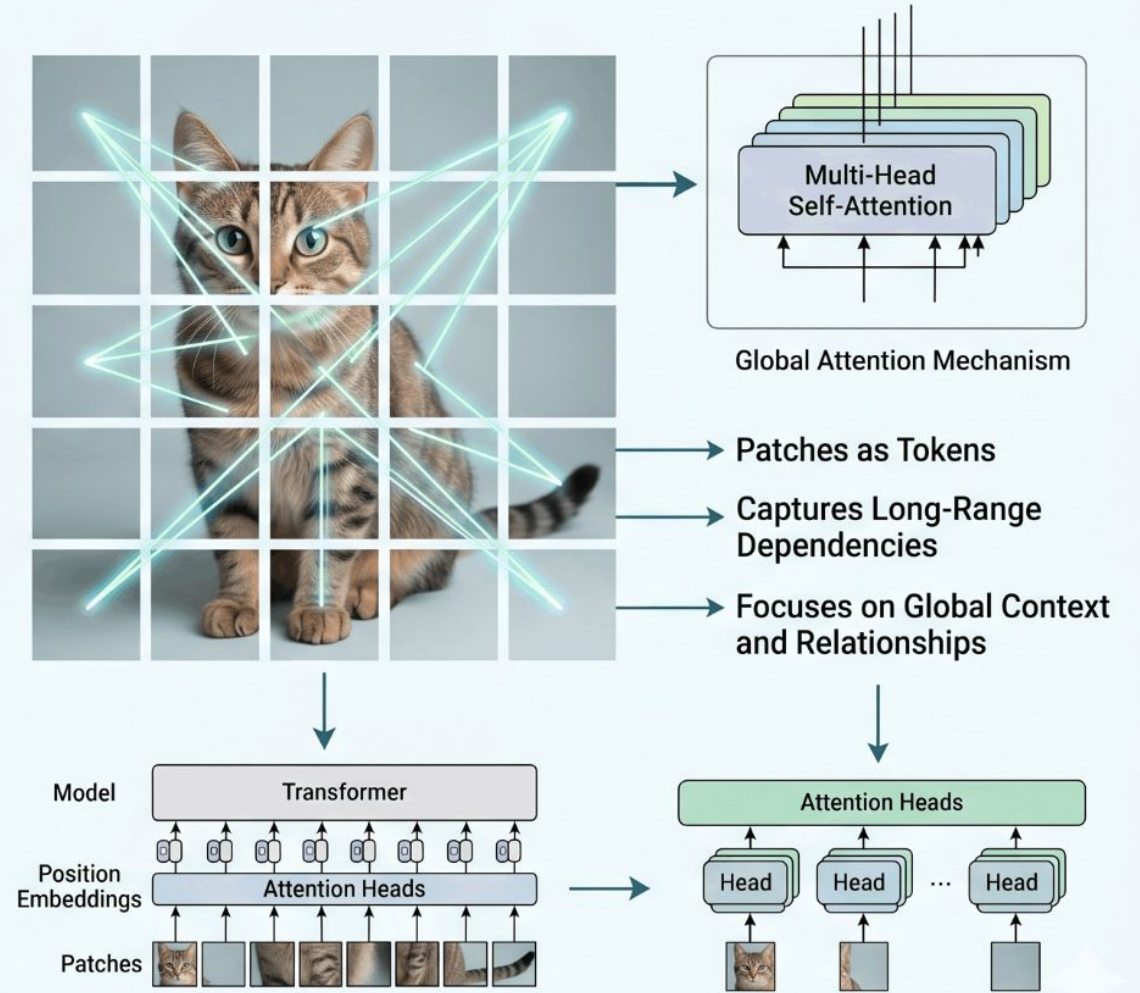
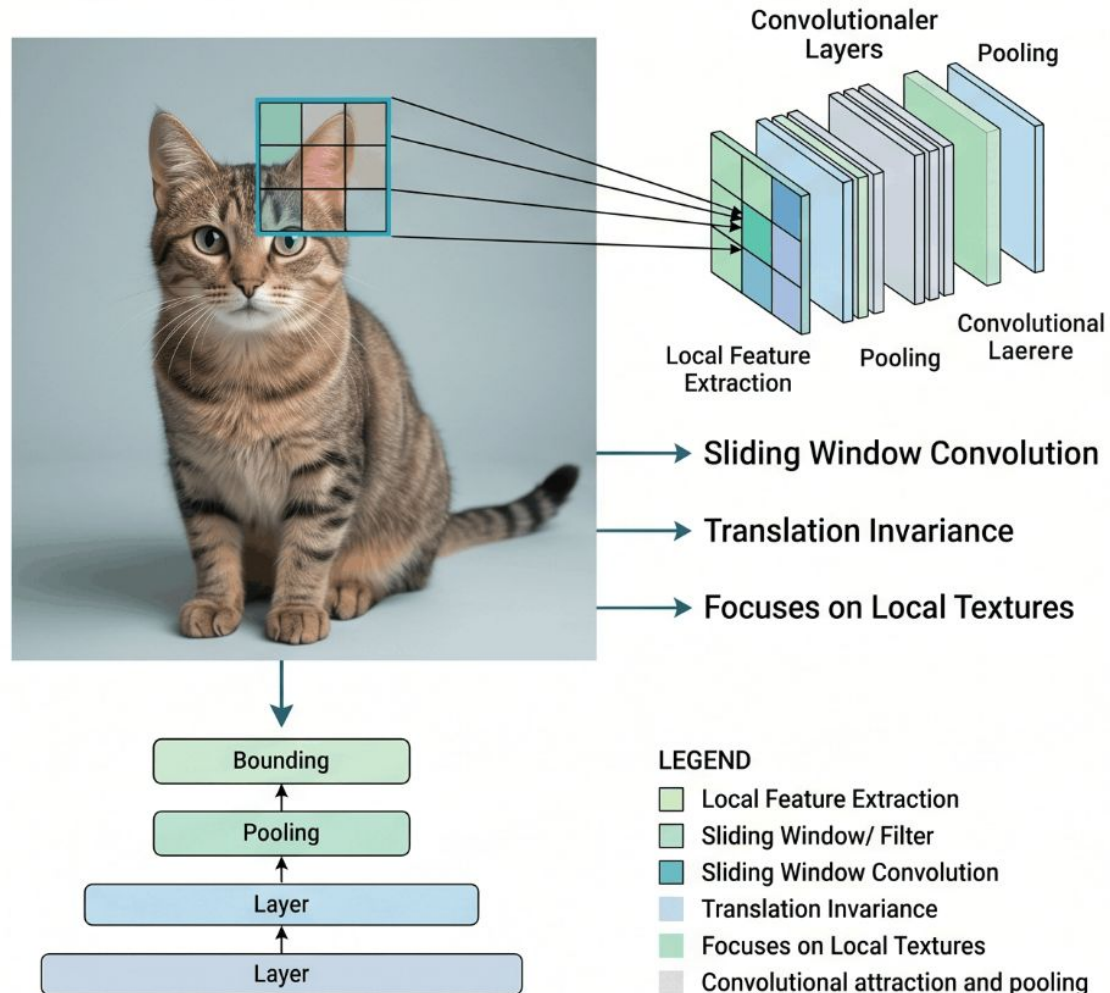
Step 3: Project encoder outputs into prediction tensor



CONVOLUTIONAL NEURAL NETWORK (CNN)

COMPARISON OF COMPUTER VISION ARCHITECTURES

VISION TRANSFORMER (ViT)





PART 03

Generative AI & GANs

From recognising what exists to creating what doesn't

From “What Is This?” to “Make Me One”

Every model so far has been discriminative — given an image, it answers a question about it (a class, a box, a mask). Generative models flip the direction entirely: given nothing but random noise, or a rough description, they produce a brand-new image that never existed before.

DISCRIMINATIVE

Image → Label

“Is this a cat?” → Yes

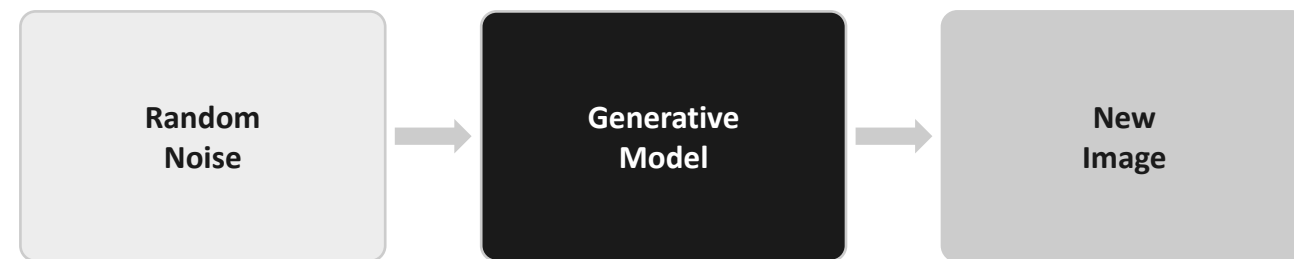
GENERATIVE

Noise → Image

“Draw me a cat” → new image

What Is Generative AI, in Plain Language?

Generative AI covers any model whose job is to produce new content — images, text, audio, or video — that resembles the data it was trained on, without copying it directly. It has learned the underlying patterns and statistics of the training data well enough to sample brand-new, plausible examples.

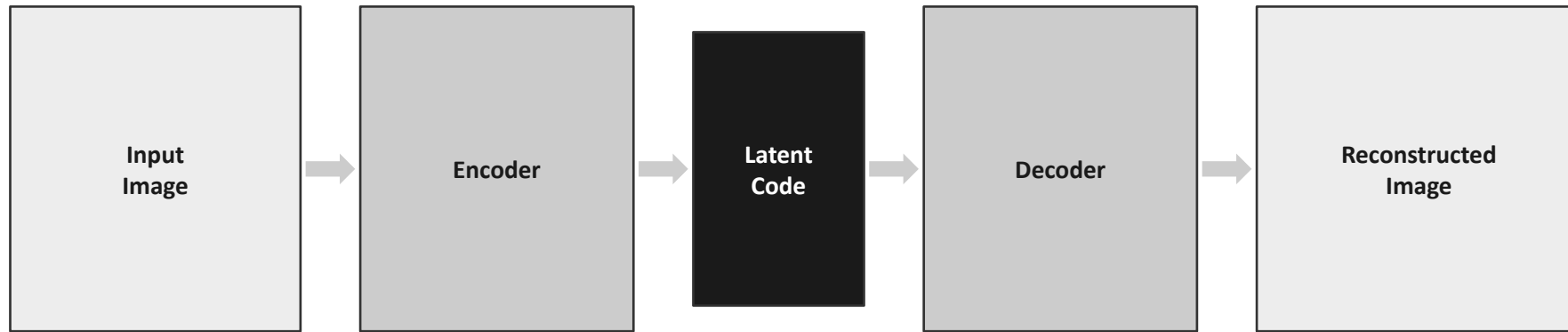


The model never saw this exact output during training — it learned the general “shape” of the data well enough to invent new, believable examples.

- Text generators: ChatGPT-style language models
- Image generators: GANs, diffusion models
- In this deck, we focus on image generation

Autoencoders: The Building Block Before GANs

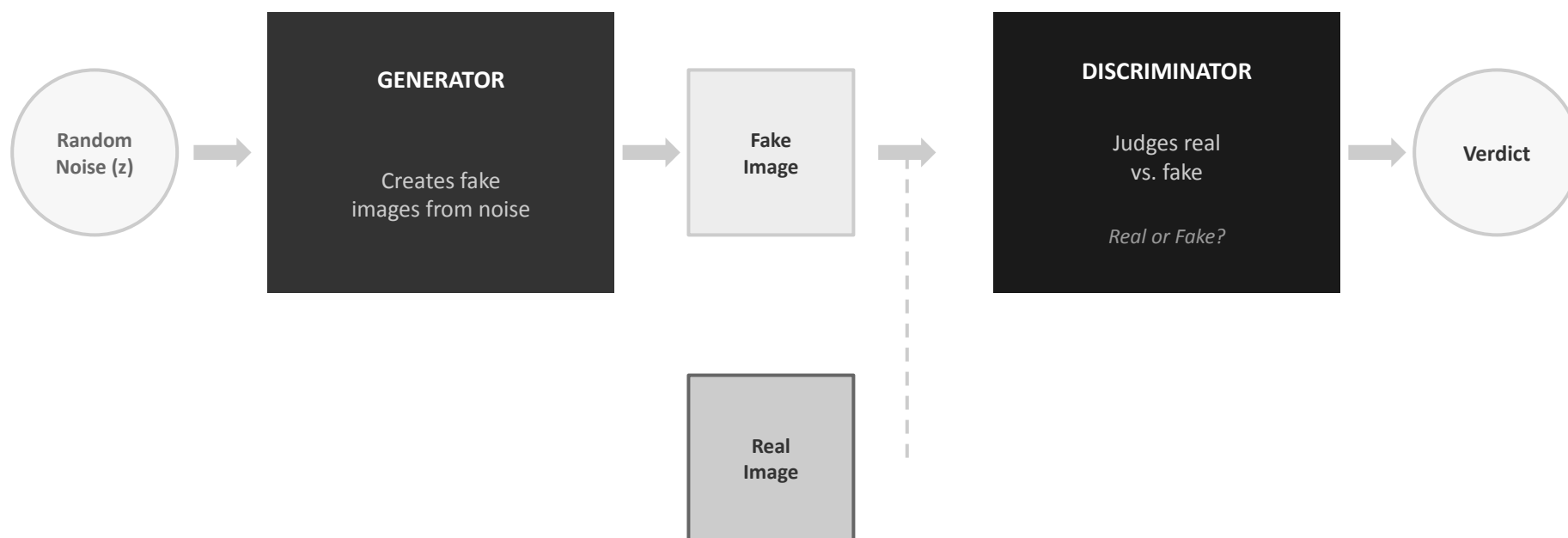
An autoencoder compresses an image down to a small “latent” code, then tries to reconstruct the original from that code alone. Squeezing through this narrow bottleneck forces the network to keep only the most essential information — the same idea GANs later build on to generate images from scratch.



THE BOTTLENECK IN THE MIDDLE IS THE KEY IDEA — IT FORCES COMPRESSION

GANs: Two Networks Competing to Improve Each Other

A Generative Adversarial Network pits two networks against one another. The Generator tries to create images realistic enough to fool the Discriminator. The Discriminator tries to correctly tell real training images apart from the Generator's fakes. Each one improves by trying to beat the other.



Feedback trains BOTH networks: the Generator improves at fooling, the Discriminator improves at spotting fakes.

How a GAN Actually Trains, Step by Step

- 1 Generator produces a batch of fake images from random noise.
- 2 Discriminator is shown a mix of real and fake images and scores each one.
- 3 Discriminator's weights update to get better at telling real from fake.
- 4 Generator's weights update to produce images that fool the Discriminator more.
- 5 Repeat thousands of times — both networks improve together until fakes look convincingly real.

Build a Simple GAN in Google Colab (MNIST digits)

GOOGLE COLAB

```
import torch, torch.nn as nn

# Generator: noise vector -> 28x28 fake digit
G = nn.Sequential(
    nn.Linear(100, 256), nn.ReLU(),
    nn.Linear(256, 784), nn.Tanh()          # 784 = 28*28 pixels
)

# Discriminator: 28x28 image -> real/fake probability
D = nn.Sequential(
    nn.Linear(784, 256), nn.LeakyReLU(0.2),
    nn.Linear(256, 1), nn.Sigmoid()
)

loss_fn = nn.BCELoss()
opt_G = torch.optim.Adam(G.parameters(), lr=2e-4)
opt_D = torch.optim.Adam(D.parameters(), lr=2e-4)
# Training loop alternates: update D, then update G, each batch
```

GAN Variants: Same Core Idea, Different Goals

Variant	Key Idea	Typical Use
DCGAN	Convolutional layers instead of plain dense layers	General image generation
CycleGAN	Learns to translate between two image domains without paired examples	Style transfer (photo ↔ painting)
Pix2Pix	Learns a mapping from a paired input image to an output image	Sketch → photo, map → satellite
StyleGAN	Separates high-level style from fine detail during generation	Photorealistic faces, high control

GANs in Action

1

Image Synthesis

Generating entirely new, realistic photos of faces, objects, or scenes.

2

Style Transfer

Repainting a photo in the style of a painting, or turning day into night.

3

Super-Resolution

Upscaling a low-quality image into a sharp, high-resolution version.

4

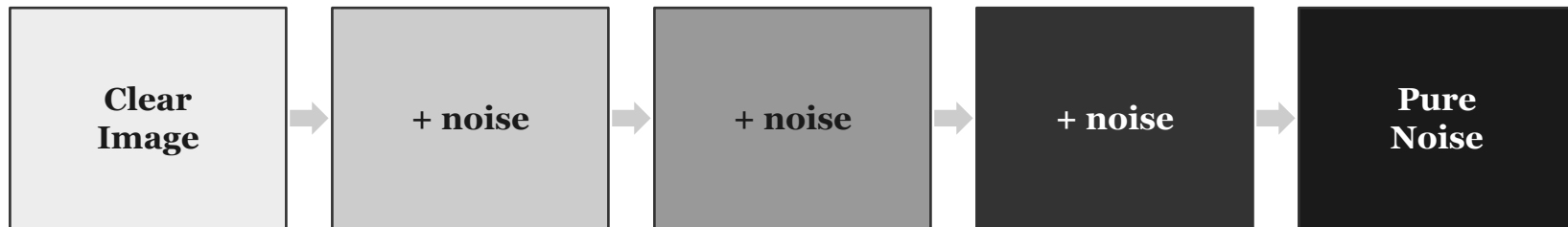
Data Augmentation

Creating extra synthetic training examples for rare classes.

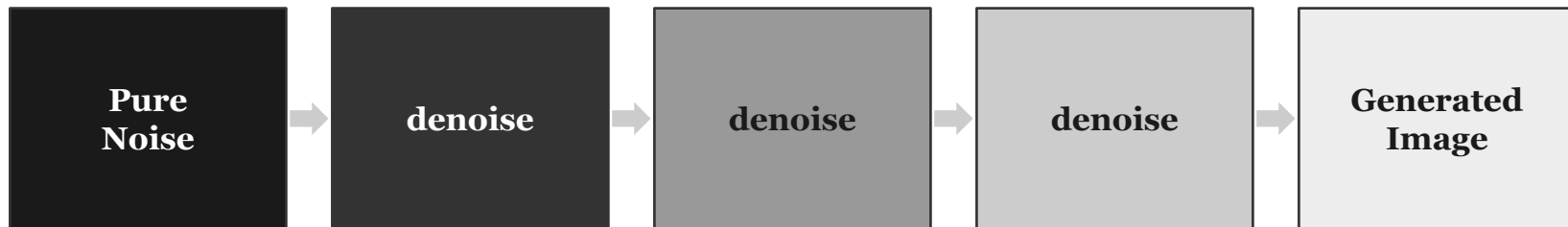
Beyond GANs: Where Generative AI Is Headed

GANs were the dominant image-generation approach for years, but training two competing networks is notoriously unstable. Diffusion models — the technology behind most modern image generators — take a different, steadier approach.

FORWARD PROCESS (TRAINING): GRADUALLY ADD NOISE



REVERSE PROCESS (GENERATION): A MODEL LEARNS TO REMOVE NOISE, STEP BY STEP



Powers tools like DALL·E, Midjourney, and Stable Diffusion — the current state of the art in image generation.

Practice: Real-Time Self-Check Quiz

Before moving on, test what stuck. The accompanying interactive quiz covers every part of this deck — from pixels and CNNs to GANs and diffusion — with instant feedback and a short explanation after each answer, so gaps show up immediately instead of at exam time.

1

Instant Feedback

Each answer is graded immediately, with a short explanation of why it's right or wrong.

2

Full-Deck Coverage

Questions span all three parts — foundations, deep learning, and generative AI.

3

Flashcard Mode

Flip the same question set into flashcards for quick pre-exam revision.

Key Takeaways

- Computer Vision turns pixel grids into meaning — every technique fits the same pipeline.
- CNNs learn their own features by stacking filters, replacing hand-designed rules.
- Detection and segmentation extend “what” to “where” and “exact shape.”
- GANs pit a Generator against a Discriminator to create realistic new images.
- Diffusion models now lead image generation, building on ideas GANs pioneered.

Resources & Next Steps

Topic	Suggested Colab Starting Point
CNN basics	PyTorch “Training a Classifier” tutorial notebook
Transfer learning	torchvision.models pretrained model zoo
Object detection	Ultralytics YOLO quick-start Colab
GANs	PyTorch DCGAN tutorial notebook (colab.research.google.com)
Diffusion models	Hugging Face “Diffusers” library quick-start notebook

All notebooks run free of charge on Google Colab — no local GPU required.